```
of grokking across different random seeds for each strategy. 9. Analyze
computational efficiency by comparing total number of examples needed to
achieve grokking for each strategy. 10. Compare attention patterns at key
points (pre-grokking, during grokking, post-grokking) across strategies to
understand how adaptive curriculum affects internal representations.",
"Interestingness": 9,
"Feasibility": 8,
"Novelty": 9,
"novel": true
```

Idea 40/50 - task_structure_grokking

```
"Name": "task_structure_grokking",
"Title": "Task Structure and Grokking: Investigating the Relationship
Between Algorithmic Complexity and Learning Dynamics",
"Experiment": "1. Modify AbstractDataset to include a
'structural_complexity' score based on: a) number of unique outputs, b)
input-output correlation, c) algebraic degree for modular operations or
cycle structure for permutations. 2. Extend existing dataset classes to
include a wider range of operations (e.g., modular addition,
multiplication, exponentiation; simple and complex permutations). 3. Run
experiments across all operations, tracking time to grokking, final
validation accuracy, and learning curve smoothness. 4. Plot grokking
metrics against structural complexity scores, comparing trends between
modular arithmetic and permutation tasks. 5. Analyze correlation between
structural complexity and grokking behavior. 6. Compare attention patterns
and gradient flows across tasks of different complexity. 7. Implement a
generalization test where models trained on simpler structures are
evaluated on more complex ones. 8. Discuss implications for neural network
learning on structured vs. unstructured tasks in general machine learning
contexts.",
"Interestingness": 9,
"Feasibility": 9,
"Novelty": 9,
"novel": true
```

Idea 41/50 - numerical_base_grokking

```
"Name": "numerical_base_grokking",
"Title": "Numerical Base and Grokking: How Input Representation Affects
Pattern Recognition in Algorithmic Learning",
"Experiment": "1. Modify AbstractDataset and modular arithmetic dataset
classes to support binary and decimal bases. 2. Implement functions to
convert between bases and adjust the encode/decode methods. 3. Update the
Transformer class to handle variable input lengths. 4. Run experiments for
binary and decimal bases on modular addition and multiplication tasks. 5.
Track metrics: time to grokking (99%
accuracy, training loss, and 'cross-base generalization' (accuracy when
testing on the other base). 6. Plot learning curves for each base,
highlighting grokking points. 7. Compare learning curves for binary (0-3)
vs decimal (0-9) to isolate base effects from sequence length. 8. Analyze
how different bases affect grokking speed and pattern recognition. 9.
Compare attention patterns across bases at key stages: pre-grokking, during
grokking, and post-grokking. 10. Discuss implications for choosing input
```

```
representations in mathematical machine learning tasks.",
"Interestingness": 9,
"Feasibility": 9,
"Novelty": 9,
"novel": true
```

Idea 42/50 - activation_function_grokking

```
"Name": "activation_function_grokking",
"Title": "Activation Functions and Grokking: Investigating the Role of Non-
linearity in Algorithmic Learning and Generalization",
"Experiment": "1. Modify the DecoderBlock class to support multiple
activation functions (ReLU, GELU, Tanh). 2. Update the Transformer class to
accept an activation_type parameter, allowing for both uniform and hybrid
activation setups. 3. Run experiments comparing the baseline (GELU) with
ReLU, Tanh, and a hybrid setup (ReLU in lower layers, Tanh in upper layers)
across all datasets. 4. Track metrics: time to grokking (99%
accuracy), final validation accuracy, training loss, 'grokking transition
sharpness', and gradient flow statistics. 5. Plot learning curves for each
activation setup, highlighting grokking points and transition periods. 6.
Visualize decision boundaries at different training stages for each
activation setup. 7. Analyze how different activation functions affect
grokking behavior, transition sharpness, and final performance for each
operation type. 8. Compare hidden representations (using t-SNE) across
activation setups at key stages: pre-grokking, during grokking transition,
and post-grokking. 9. Investigate the relationship between activation
function properties and the trade-off between memorization and
generalization. 10. Discuss implications for choosing activation functions
in tasks requiring pattern discovery and generalization beyond algorithmic
learning.",
"Interestingness": 9,
"Feasibility": 9,
"Novelty": 9,
"novel": true
```

Idea 43/50 - phase_transition_grokking

```
"Name": "phase_transition_grokking",
"Title": "Grokking as a Phase Transition: Characterizing Critical Behavior
in Algorithmic Learning",
"Experiment": "1. Implement functions to track key metrics: validation
accuracy, training loss, gradient norm, and weight norm. 2. Modify training
loop to compute and store these metrics every 100 steps. 3. Run experiments
across all datasets, with finer-grained tracking (every 10 steps) around
the suspected grokking point. 4. Implement analysis tools to detect sudden
changes or discontinuities in metrics. 5. Plot all metrics on a single,
multi-axis graph to visualize potential phase transitions. 6. Calculate
susceptibility using fluctuations in validation accuracy near the grokking
point. 7. Analyze scaling behavior of susceptibility to identify critical
exponents, if any. 8. Compare phase transition characteristics across
different operations and model sizes. 9. Investigate whether manipulating
learning rate or gradient clipping can induce or prevent grokking phase
transitions.",
"Interestingness": 9,
```

```
"Feasibility": 8,
"Novelty": 9,
"novel": false
```


Idea 44/50 - effective_dimension_grokking


```
"Name": "effective_dimension_grokking",
"Title": "Effective Dimension Dynamics in Grokking: Analyzing
Representational Complexity During Algorithmic Learning",
"Experiment": "1. Implement functions to compute the rank and top-k
singular values of weight matrices. 2. Modify the training loop to compute
and store these metrics every 500 steps for each layer. 3. Run experiments
across all datasets, tracking rank and singular value distributions
alongside existing performance metrics. 4. Implement a simple MLP baseline
that doesn't exhibit grokking for comparison. 5. Plot the evolution of rank
and singular value distributions alongside grokking curves for both
Transformer and MLP models. 6. Analyze how these metrics change before,
during, and after grokking in the Transformer, contrasting with the MLP. 7.
Compare rank dynamics between operations that grok quickly vs. slowly. 8.
Investigate correlations between changes in rank/singular values and
grokking speed or generalization performance. 9. Visualize the relationship
between these metrics and other performance indicators at different stages
of training.",
"Interestingness": 9,
"Feasibility": 8,
"Novelty": 9,
"novel": true
```


Idea 45/50 - representation_entropy_grokking


```
"Name": "representation_entropy_grokking",
"Title": "Representation Entropy in Grokking: Tracking the Simplification
of Learned Concepts",
"Experiment": "1. Implement a function to compute the entropy of the
model's internal representations. 2. Modify the Transformer class to output
intermediate representations. 3. Update the training loop to compute and
store the representation entropy every 500 steps. 4. Run experiments across
all datasets, including configurations that lead to successful grokking and
those that don't (e.g., by varying model size or learning rate). 5. Track
entropy alongside existing performance metrics. 6. Plot the evolution of
representation entropy alongside grokking curves for both successful and
unsuccessful cases. 7. Analyze how representation entropy changes before,
during, and after grokking in successful cases, and compare with
unsuccessful cases. 8. Investigate correlations between changes in
representation entropy and grokking speed or generalization performance. 9.
Visualize the relationship between entropy and other performance indicators
at different stages of training. 10. Plot entropy distributions across
different layers of the model to understand how different parts contribute
to concept simplification.",
"Interestingness": 9,
"Feasibility": 8,
"Novelty": 8,
"novel": true
```